



Name: Shayan Umar

Designation: AI intern

Department: Artificial Intelligence

Date: 22-8-2025

Table of Contents

1. Theoretical Understanding:	3
1.1 Clustering:	3
1.2 Generative Adversarial Networks (GANs):	9
2. Project Work:	17
Task01: Loan Data Analysis & Prediction	17
Task02: Book Recommender	18
3. Challenges and Approaches:	19
Task 1: Loan Default Prediction (Decision Tree & Random Forest)	19
Task 2: Book Recommender (k-NN)	20
4. Result and Analysis:	21
Task01: Loan Data Analysis & Prediction	21
Task02: Book Recommender	23
5. Conclusion:	26
6. References:	27

1. Theoretical Understanding:

1.1 Clustering:

Clustering in unsupervised machine learning is the process of grouping unlabeled data into clusters based on their similarities. The goal of clustering is to identify patterns and relationships in the data without any prior knowledge of the data's meaning.

Broadly this technique is applied to group data based on different patterns, such as similarities or differences, our machine model finds. These algorithms are used to process raw, unclassified data objects into groups. For example, in the above figure, we have not given output parameter values, so this technique will be used to group clients based on the input parameters provided by our data.

Some common clustering algorithms:

- **K-means Clustering**: Groups data into K clusters based on how close the points are to each other.
- **Hierarchical Clustering**: Creates clusters by building a tree step-by-step, either merging or splitting groups.
- **Density-Based Clustering (DBSCAN)**: Finds clusters in dense areas and treats scattered points as noise.
- **Mean-Shift Clustering**: Discovers clusters by moving points toward the most crowded areas.
- **Spectral Clustering**: Groups data by analyzing connections between points using graphs.

K means Clustering

K-Means Clustering is an Unsupervised Machine Learning algorithm which groups unlabeled dataset into different clusters. It is used to organize data into **groups based on their similarity**.

For example online store uses K-Means to group customers based on purchase frequency and spending creating segments like Budget Shoppers, Frequent Buyers and Big Spenders for personalised marketing.

The algorithm works by first randomly picking some central points called **centroids** and each data point is then assigned to the closest centroid forming a cluster. After all the points are assigned to a cluster the centroids are updated by finding the average position of the points in each cluster. This process repeats until the centroids stop changing forming clusters. The goal of clustering is to divide the data points into clusters so that similar data points belong to same group.

How k-means clustering works?

We are given a data set of items with certain features and values for these features like a vector. The task is to categorize those items into groups. To achieve this we will use the K-means algorithm. 'K' in the name of the algorithm represents the number of groups/clusters we want to classify our items into.

The algorithm will categorize the items into k groups or clusters of similarity. To calculate that similarity we will use the Euclidean distance as a measurement. The algorithm works as follows:

1. First we randomly initialize k points called means or cluster centroids.
2. We categorize each item to its closest mean and we update the mean's coordinates, which are the averages of the items categorized in that cluster so far.
3. We repeat the process for a given number of iterations and at the end, we have our clusters.

Selecting the right number of clusters is important for meaningful segmentation to do this we use Elbow Method for optimal value of k in KMeans which is a graphical tool used to determine the optimal number of clusters (k) in K-means.

Final Analogy Summary:

Think of the entire process like:

1. **Scattering dots** on a board.

2. Dropping **magnets** randomly.
3. Each dot goes to its nearest magnet.
4. Each magnet moves to the **average location** of its attracted dots.
5. Repeat until magnets **settle down** (not shown here, but needed for full K-means).
6. Plot the final "teams" of dots and their leaders.

Neural Nets for Clustering:

Autoencoders, Predictors and Embeddings

What is a Supervised Similarity Measure?

- Instead of manually engineering features and comparing them directly, we use a **deep neural network (DNN)** to learn **embeddings** (vector representations).
- These embeddings place similar examples close together in an **embedding space**.
- Example: YouTube videos watched by the same users → embeddings for those videos end up close together.
- The "closeness" (via cosine similarity, dot product, or Euclidean distance) becomes the **similarity measure**.

The supervised step is **only to create the similarity measure**. Once learned, the similarity can be used for **unsupervised clustering** or retrieval.

Manual vs Supervised Measures

Requirement	Manual Similarity	Supervised Similarity
Eliminates redundant info in correlated features?	✗ No	✓ Yes, DNN compresses
Interpretability (insight into similarities)?	✓ Yes	✗ No, embeddings are opaque
Small datasets, few features	✓ Suitable	✗ Not enough data for DNN

Requirement	Manual Similarity	Supervised Similarity
Large datasets, many features	✗ Hard to engineer	✓ DNN handles automatically

How to Create a Supervised Similarity Measure

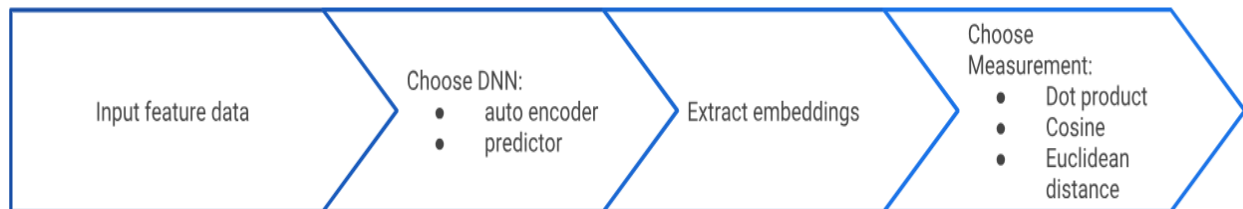


Figure 1: Steps to create a supervised similarity measure.

1. **Input feature data**
2. **Choose DNN type:**
 - **Autoencoder** → predicts all input features.
 - **Predictor** → predicts only an important feature.
3. **Train the network.**
4. **Extract embeddings** (from the bottleneck/embedding layer).
5. **Choose similarity metric:** dot product, cosine, or Euclidean distance.

DNN Choices

1. Autoencoder

- Learns embeddings by trying to **reconstruct the input data**.
- Has a bottleneck (hidden layer smaller than input/output).
- Embeddings = compressed representation at the bottleneck layer.
- Best when **all features are equally important**.

2. Predictor

- Learns embeddings by trying to **predict a chosen feature** (instead of all features).

- Useful when some features are **more important than others** (e.g., house price > postal code).
- Embeddings usually extracted from the **last embedding layer** before the output.
- ⚠ Make sure to **remove the label feature** from the input to avoid **label leakage**.

Key Takeaway

- **Autoencoder** = unsupervised compression → good for balanced features.
- **Predictor DNN** = supervised compression → good when one feature dominates similarity.
- Once trained, embeddings + distance metric = a **powerful similarity measure** for clustering or retrieval.

Measuring similarity from embeddings

You now have embeddings for any pair of examples. A supervised similarity measure takes these embeddings and returns a number measuring their similarity. Remember that embeddings are vectors of numbers. To find the similarity between two vectors a and b , choose one of these three similarity measures:

Measure	Meaning	Formula	As similarity increases, this measure...
Euclidean distance	Distance between ends of vectors	$\sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_N - b_N)^2}$	Decreases
Cosine	Cosine of angle θ between vectors	$\frac{a^T b}{ a \cdot b }$	Increases
Dot product	Cosine multiplied by lengths of both vectors	$a_1 b_1 + a_2 b_2 + \dots + a_n b_n = a b \cos(\theta)$	Increases. Also increases with length of vectors.

Choosing a similarity measure

In contrast to the cosine, the dot product is proportional to the vector length. This is important because examples that appear very frequently in the training set (for example, popular YouTube videos) tend to have embedding vectors with large lengths. If you want to capture popularity, then choose dot product. However, the risk is that popular examples may skew the similarity metric. To balance this skew, you can raise the length to an exponent to calculate the dot product as $|a|^p |b|^p \cos(\theta)$.

Proof: Proportionality of Similarity Measures

After normalizing a and b such that $||a|| = 1$ and $||b|| = 1$, these three measures are related as:

- Euclidean distance = $||a - b|| = \sqrt{||a||^2 + ||b||^2 - 2a^T b} = \sqrt{2 - 2 \cos(\theta_{ab})}$.
- Dot product = $|a||b| \cos(\theta_{ab}) = 1 \cdot 1 \cdot \cos(\theta_{ab}) = \cos(\theta_{ab})$.
- Cosine = $\cos(\theta_{ab})$.

Thus, all three similarity measures are equivalent because they are proportional to $\cos(\theta_{ab})$.

1.2 Generative Adversarial Networks (GANs):

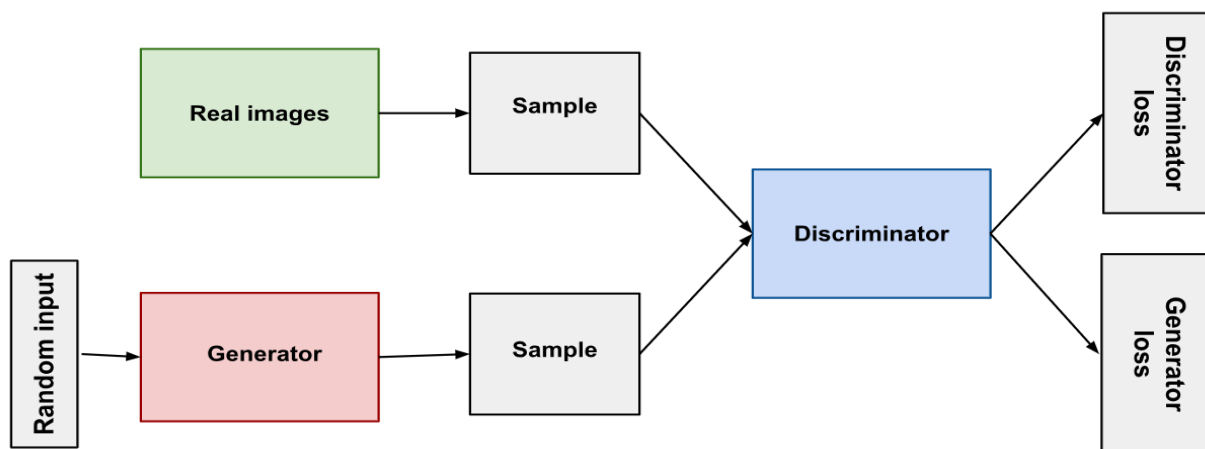
What is a GAN?

A **Generative Adversarial Network (GAN)** is a machine learning model introduced by Ian Goodfellow in 2014.

It is made of **two neural networks** that compete with each other:

- **Generator (G)**: tries to create fake data that looks real.
- **Discriminator (D)**: tries to distinguish between real data (from the dataset) and fake data (from the generator).

This “adversarial” setup makes the generator improve over time until it produces very realistic outputs.



How it works

1. Start with **real data samples** (e.g., real images) and **random noise** as input to the generator.
2. The **generator** converts random noise into a fake sample (e.g., a fake image).
3. The **discriminator** receives both real and fake samples and tries to classify them correctly.
4. If the discriminator is fooled, the generator learns that it is improving.
5. This process continues, making both models better:
 - The generator learns to make realistic data.

- The discriminator learns to detect fakes more accurately.

Example analogy

- The **generator** is like a counterfeiter making fake currency.
- The **discriminator** is like the police trying to detect counterfeits.
- Over time, the counterfeiter improves at making realistic money, while the police get better at spotting fakes.
- Eventually, the fake becomes almost indistinguishable from the real.

Generative models imitate data distributions by producing convincing new examples.

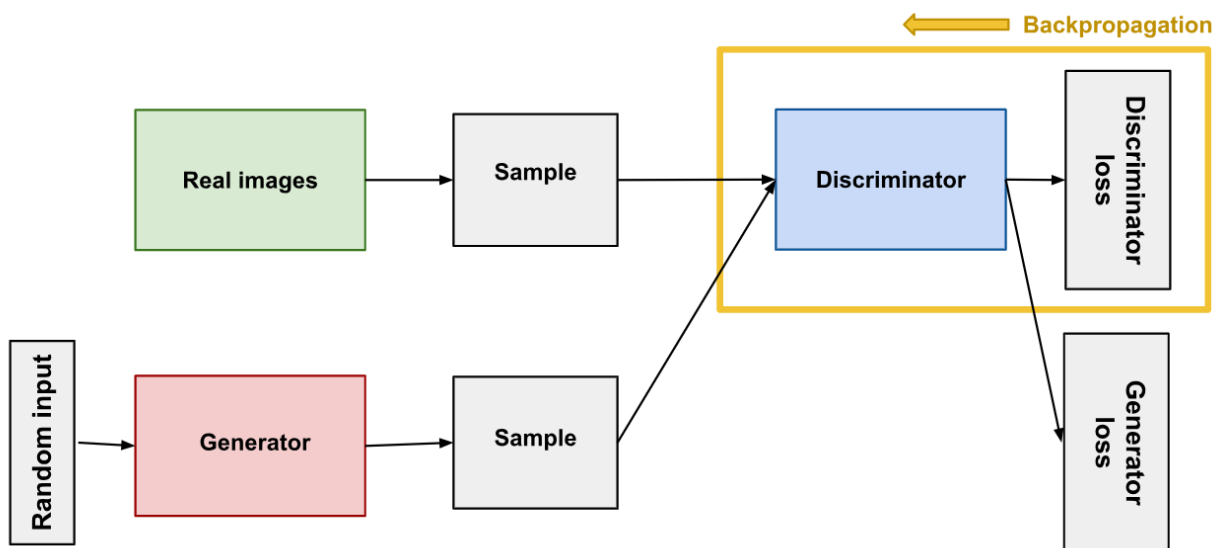
Discriminative models only separate classes, without modeling how data itself is distributed.

Generative modeling is harder, since it must capture complex correlations (e.g., boats with water, faces with eyes in the right place).

Discriminative models can often succeed with fewer correlations, just finding decision boundaries.

Discriminator:

The discriminator in a GAN is simply a classifier. It tries to distinguish real data from the data created by the generator. It could use any network architecture appropriate to the type of data it's classifying.



Discriminator Training Data

The discriminator's training data comes from two sources:

- **Real data** instances, such as real pictures of people. The discriminator uses these instances as positive examples during training.
- **Fake data** instances created by the generator. The discriminator uses these instances as negative examples during training.

Training the Discriminator

The discriminator connects to two loss functions. During discriminator training, the discriminator ignores the generator loss and just uses the discriminator loss. We use the generator loss during generator training.

During discriminator training:

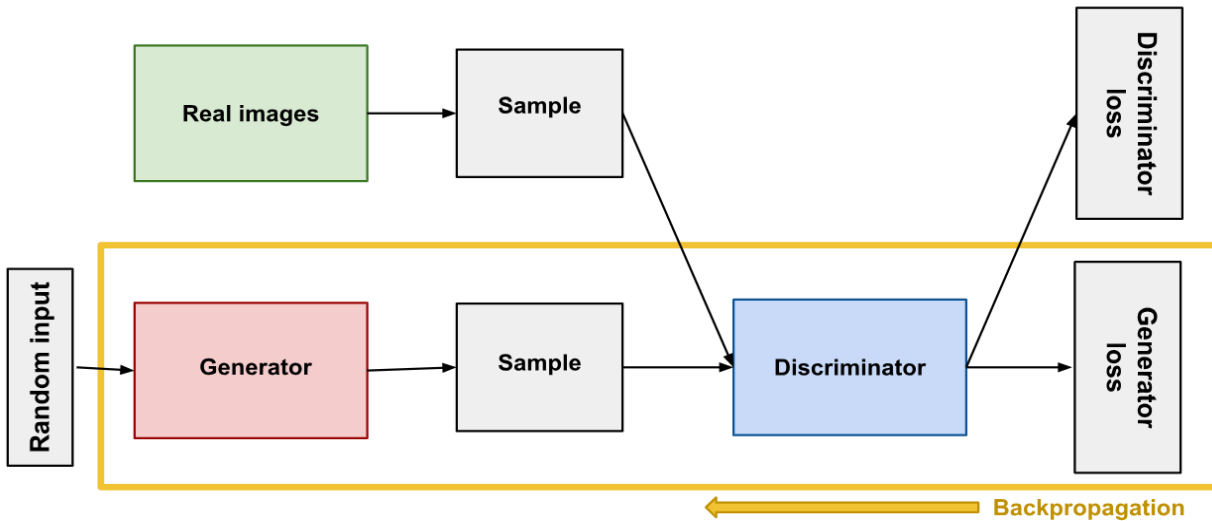
1. The discriminator classifies both real data and fake data from the generator.
2. The discriminator loss penalizes the discriminator for misclassifying a real instance as fake or a fake instance as real.
3. The discriminator updates its weights through backpropagation from the discriminator loss through the discriminator network.

Generator

The generator part of a GAN learns to create fake data by incorporating feedback from the discriminator. It learns to make the discriminator classify its output as real.

Generator training requires tighter integration between the generator and the discriminator than discriminator training requires. The portion of the GAN that trains the generator includes:

- random input
- generator network, which transforms the random input into a data instance
- discriminator network, which classifies the generated data
- discriminator output
- generator loss, which penalizes the generator for failing to fool the discriminator



Random Input for GANs

- A GAN's **generator** starts with **random noise** as input.
- The noise is usually sampled from an easy distribution (e.g., uniform).
- This randomness lets the generator produce diverse outputs, covering different parts of the data distribution.
- Noise is usually lower-dimensional than the output, making sampling efficient.
- Some GAN variations use structured (non-random) input to guide outputs.

Training the Generator with the Discriminator

- The **generator doesn't have a direct loss**; it relies on the **discriminator's feedback**.
- Process:
 1. Sample random noise.
 2. Generator produces fake data.
 3. Discriminator classifies it as real/fake.
 4. Generator's loss penalizes it if the discriminator detects "fake."
 5. Backpropagation flows **through the discriminator into the generator**, updating only the generator's weights.
- Important: During generator training, the discriminator's weights are **frozen** so the generator isn't chasing a moving target.

In short: **GANs use random noise as input to create variety, and the generator learns by backpropagating through the discriminator's judgment of "real vs. fake."**

GANs Training:

Alternating Training

- **Discriminator training phase:**
 - Generator is frozen.
 - Discriminator learns to tell real data from fake (based on the generator's current flaws).
- **Generator training phase:**
 - Discriminator is frozen.
 - Generator improves based on discriminator feedback.
- This **back-and-forth** process makes GAN training possible.
- If the discriminator can't tell apart real vs. random fake data at the start, GAN training cannot even begin.

Convergence in GANs

- As the **generator improves**, the **discriminator gets worse** because fakes look more realistic.
 - Perfect generator $\Rightarrow$ discriminator accuracy drops to 50% (random guessing).
 - Problem: once discriminator feedback becomes random, the generator may start training on meaningless signals, leading to collapse.
 - **Convergence is unstable** and often temporary — GANs don't settle into a clear "final trained state."
-

In short: **GANs are trained in alternating steps between discriminator and generator, but convergence is tricky because the discriminator's feedback weakens as the generator improves.**

Loss Functions:

Why Loss Functions Matter in GANs

- GANs aim to replicate a **probability distribution**.
- Loss functions measure the **distance between real data distribution and generated data distribution**.
- Both the **generator** and **discriminator** have loss functions derived from the same distance measure.
- Generator only influences the part of the formula related to **fake data**.

1. Minimax Loss (original GAN paper, 2014)

- Discriminator maximizes: $\mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log(1 - D(G(z)))]$
- Generator minimizes: $\log(1 - D(G(z)))$
- **Problem:** Can cause training stalls early on (discriminator too strong).

Modified Minimax Loss:

- Instead of minimizing $\log(1 - D(G(z)))$ the generator **maximizes** $\log D(G(z))$.
- This helps avoid vanishing gradients in early training.

2. Wasserstein Loss (WGAN, 2017)

- Discriminator is renamed a **critic** (since it doesn't output probabilities).
- **Critic Loss:**

$$D(x) - D(G(z))$$

(maximize the gap between real and fake outputs).

- **Generator Loss:**

$$D(G(z))$$

(maximize critic's score for fake instances).

- Based on **Earth Mover's Distance** (true metric between distributions).
- Requires **weight clipping** to enforce constraints.

Benefits of Wasserstein loss:

- More stable training.
 - Avoids vanishing gradients.
 - Less prone to collapse than minimax-based GANs.
-

Key Takeaways

- **Minimax loss:** cross-entropy based; works but unstable, often needs modification.
- **Wasserstein loss:** more stable, uses earth mover distance, better suited for GAN training.

Common Problems:

1. Vanishing Gradients

- **Cause:** If the discriminator becomes too strong, the generator gets no useful feedback (gradients vanish).
 - **Result:** Generator training stalls.
 - **Fixes:**
 - **Wasserstein loss** (avoids vanishing gradients, even with optimal discriminator).
 - **Modified minimax loss** (proposed in original GAN paper).
-

2. Mode Collapse

- **Cause:** Generator finds one (or a few) outputs that consistently fool the discriminator and keeps repeating them.

- **Result:** Lack of diversity in generated outputs.
 - **Fixes:**
 - **Wasserstein loss** (more stable discriminator training reduces collapse).
 - **Unrolled GANs** (consider future discriminator responses, not just the current one, so generator can't overfit to a single discriminator).
-

3. Failure to Converge

- **Cause:** Training oscillates or destabilizes, with generator and discriminator failing to reach balance.
 - **Result:** GAN never settles into producing consistently realistic samples.
 - **Fixes (research approaches):**
 - Add **noise** to discriminator inputs.
 - Apply **regularization** (e.g., penalizing discriminator weights).
-

Key Takeaway

GANs are powerful but unstable.

- **Vanishing gradients** = no learning.
- **Mode collapse** = low diversity.
- **Non-convergence** = training never stabilizes.

Researchers use **Wasserstein loss, unrolled GANs, and regularization** as remedies, but no universal solution exists yet.

2. Project Work:

Task01: Loan Data Analysis & Prediction

1. Task Description

The goal is to **predict whether a loan will be fully paid or not (not.fully.paid)** using borrower financial and loan information.

We compare **Decision Tree** and **Random Forest** models, evaluate their performance, and identify key features driving loan defaults.

2. Methodology

1. Data Loading & Cleaning

- Dataset: loan_data.csv with **9,578 rows × 14 columns**.
- No missing or duplicate values found.

2. Exploratory Data Analysis (EDA)

- Histograms of FICO scores by credit.policy and not.fully.paid.
- Countplot of loan purposes vs. repayment status.
- Correlation heatmap (RdYlGn colormap).

3. Feature Engineering

- Converted purpose column into dummy variables using **OneHotEncoder** (drop='first').
- Concatenated with dataset → 6 new columns added.

4. Class Imbalance Handling

- Target distribution: 0 (fully paid) = 8045, 1 (not fully paid) = 1533.
- Used **SMOTE** oversampling to balance classes.

5. Model Training & Evaluation

- Models: **Decision Tree & Random Forest**.
- Evaluation metrics: Accuracy, Precision, Recall, F1-score.
- Performed **5-fold cross-validation** for Random Forest.

6. Model Saving

- Saved Random Forest model with **pickle** for reproducibility and deployment.

Task02: Book Recommender

Task Description: Build a Book Recommendation System using k-Nearest Neighbors (kNN) that, given a book title, returns a list of similar books with their authors, average rating, and ratings count.

1. Load and explore dataset

- Used `pd.read_csv()` with `on_bad_lines='skip'`.
- Printed shape, columns, and explored with `head()`, `describe()`, `info()`.
- This gives insight into data size, missing values, and feature types.

2. Missing values

- Confirmed with `df.isnull().sum()`.
- Critical because missing values break distance calculations in k-NN.

3. EDA

- Excellent visualizations:
 - Top books with >1M ratings.
 - Top authors, publishers, languages.
 - Distribution of ratings.
 - Boxplot by language.
- Helps identify trends and outliers in book popularity.

4. Feature Engineering

- Created `rating_between` bins.
- One-hot encoded `language_code` and `rating_between`.
- Reported number of dummy columns added.

5. Scaling

- Applied `MinMaxScaler` to avoid feature dominance (`ratings_count` >> `average_rating`).
- Essential for distance-based models like k-NN.

6. Model (k-NN with Ball Tree)

- BallTree works better for high-dimensional data than brute-force.

7. Recommender Function

- Built BookRecommender() with tabulate → clean, professional output.
- Tested with *The Hobbit* and *Pride and Prejudice*.
- Recommendations are reasonable because similar ratings/language/ratings_count group together.

8. Distance Visualization

- Plotted distances to nearest neighbors.
- If distances grow sharply → neighbors are less similar → recommendation quality drops.

9. Pickle Saving

- Saved knn, scaler, features, and df.
- Important for **reproducibility** (no need to retrain each time).
- Successfully reloaded and used for recommendations.

3. Challenges and Approaches:

Task 1: Loan Default Prediction (Decision Tree & Random Forest)

Challenges:

- Handling categorical variables and encoding them properly.
- Risk of **overfitting** with Decision Trees.
- Ensuring the model generalizes well.
- **Class imbalance** (not.fully.paid = 1 was the minority class).

Approaches:

- Used **Decision Tree** for interpretability.
- Applied **Random Forest** to reduce variance and improve performance.
- Evaluated using **confusion matrix** and **recall** to focus on the minority class.

- Feature importance identified key drivers (e.g., purpose_debt_consolidation, int.rate, credit.policy).
-

Task 2: Book Recommender (k-NN)

Challenges:

- Sparse dataset with many missing ratings.
- Cold-start problem for new users/books.
- Computational cost of k-NN for large datasets.

Approaches:

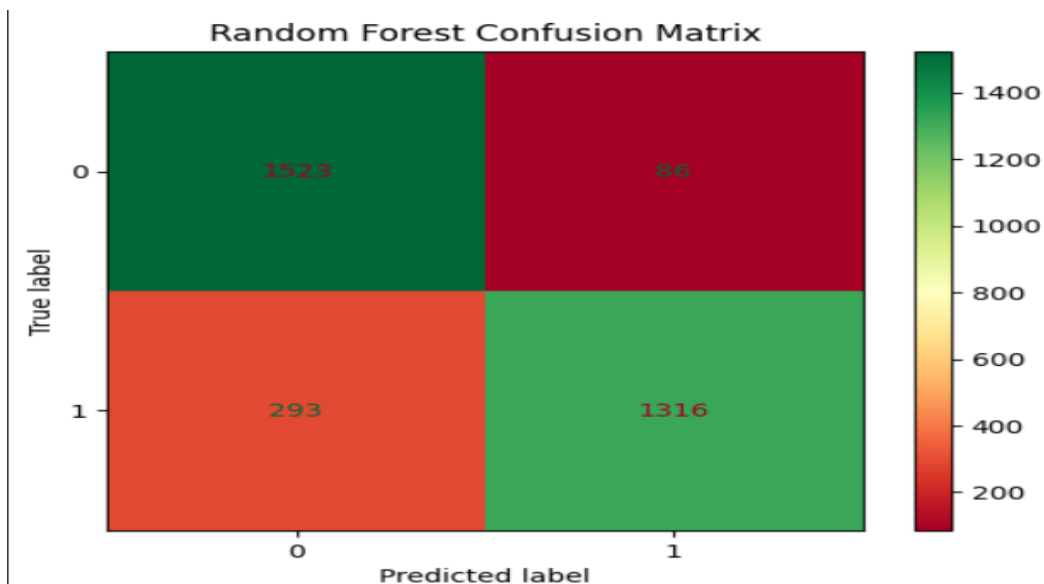
- Used **k-Nearest Neighbors (k-NN)** for collaborative filtering.
- Preprocessed by **encoding users/books** and handling missing values.
- Handled imbalance using **SMOTE oversampling** (to balance classes).

4. Result and Analysis:

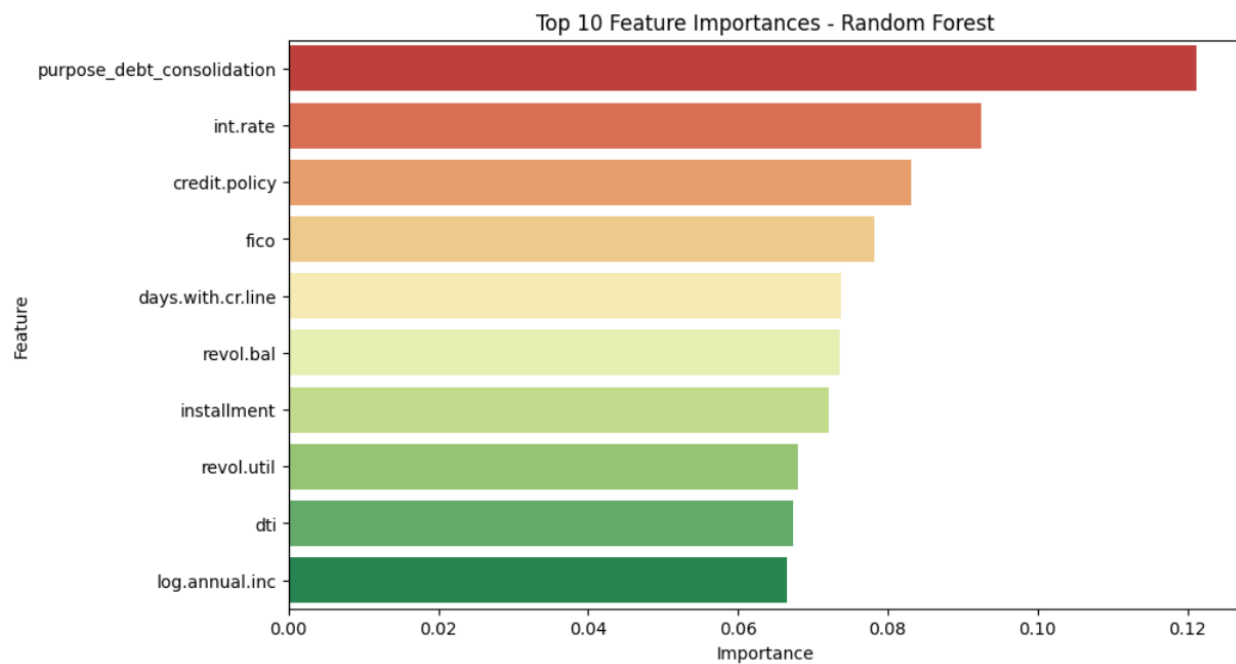
Task01: Loan Data Analysis & Prediction

```
Random Forest Performance:  
Accuracy: 0.882224984462399  
Precision: 0.9386590584878745  
Recall: 0.8178993163455562  
F1-Score: 0.8741281966124211
```

```
Classification Report:  
              precision    recall  f1-score   support  
  
    0       0.84         0.95         0.89         1609  
    1       0.94         0.82         0.87         1609  
  
 accuracy          0.88         0.88         0.88         3218  
 macro avg         0.89         0.88         0.88         3218  
 weighted avg      0.89         0.88         0.88         3218
```

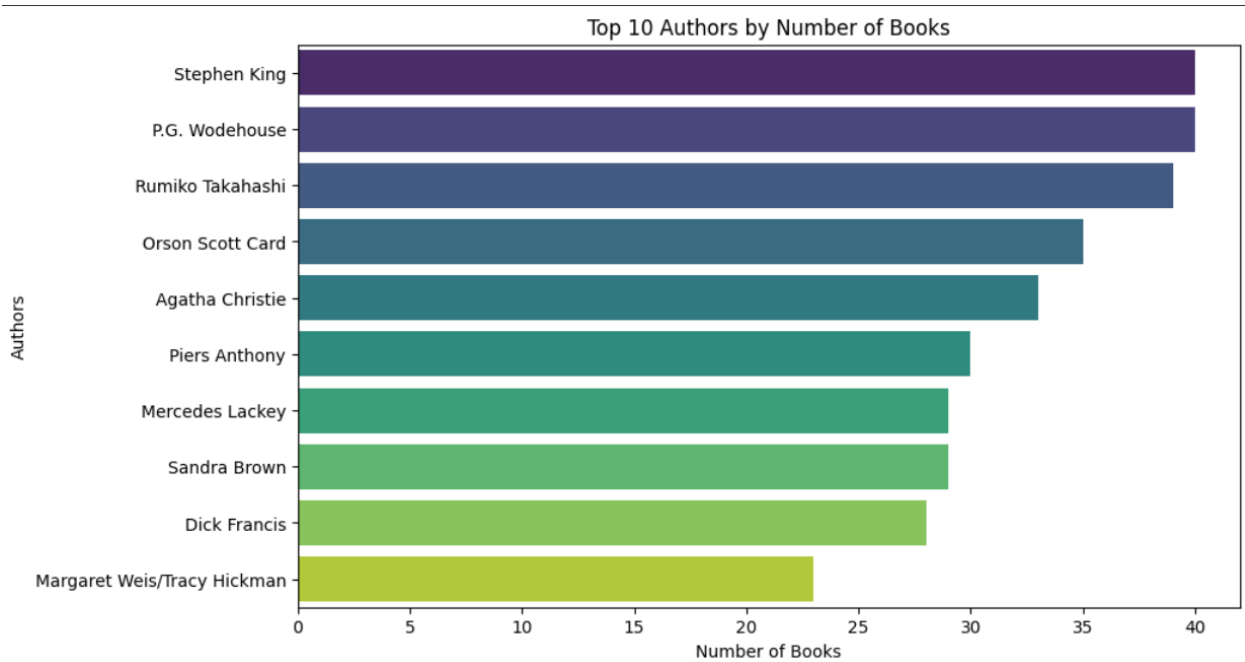
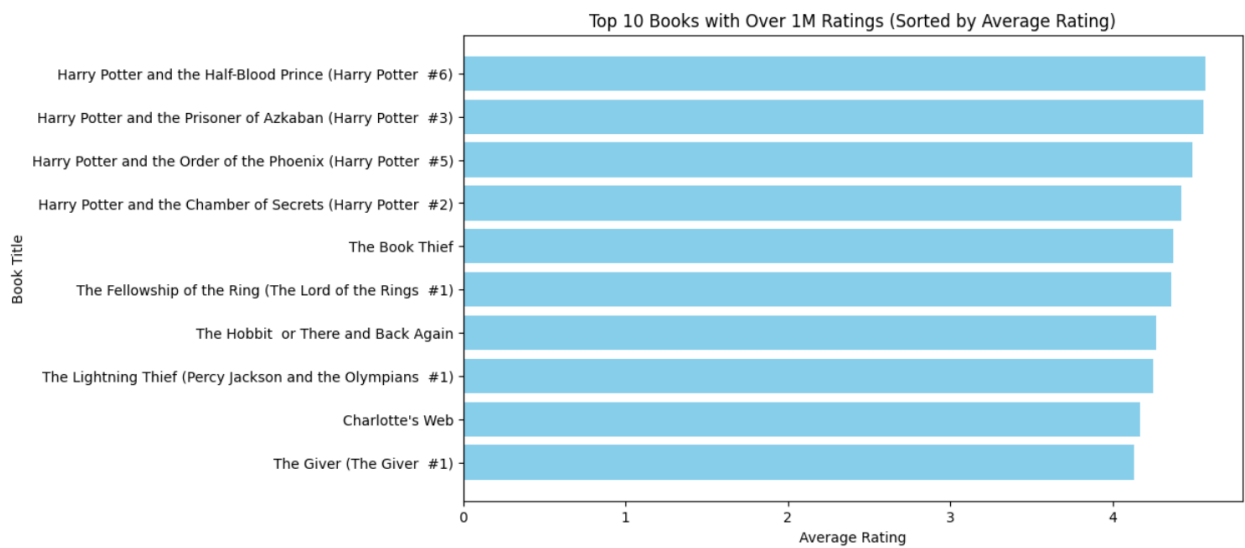


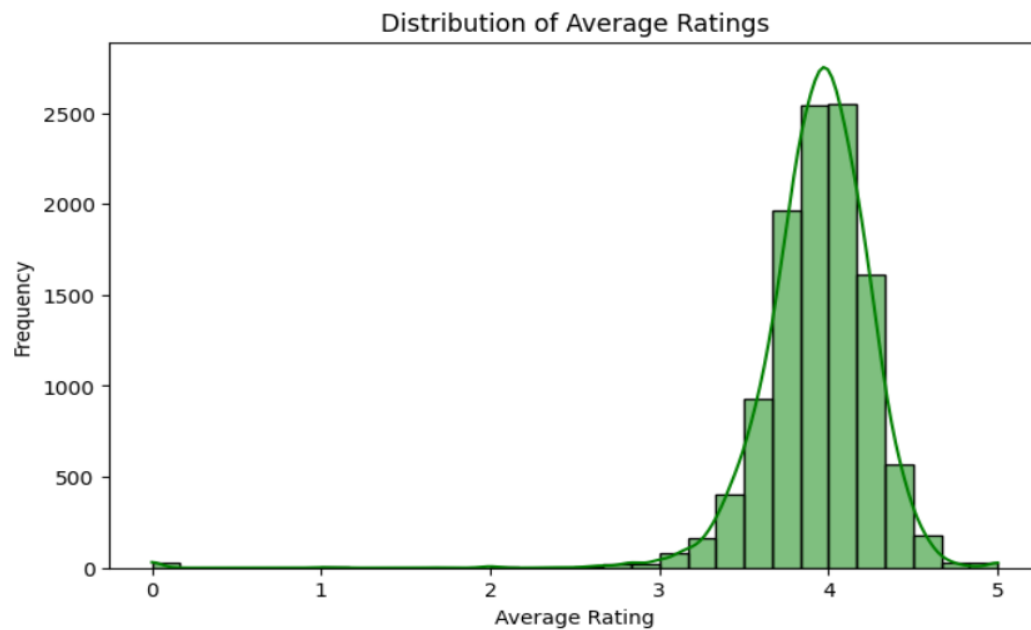
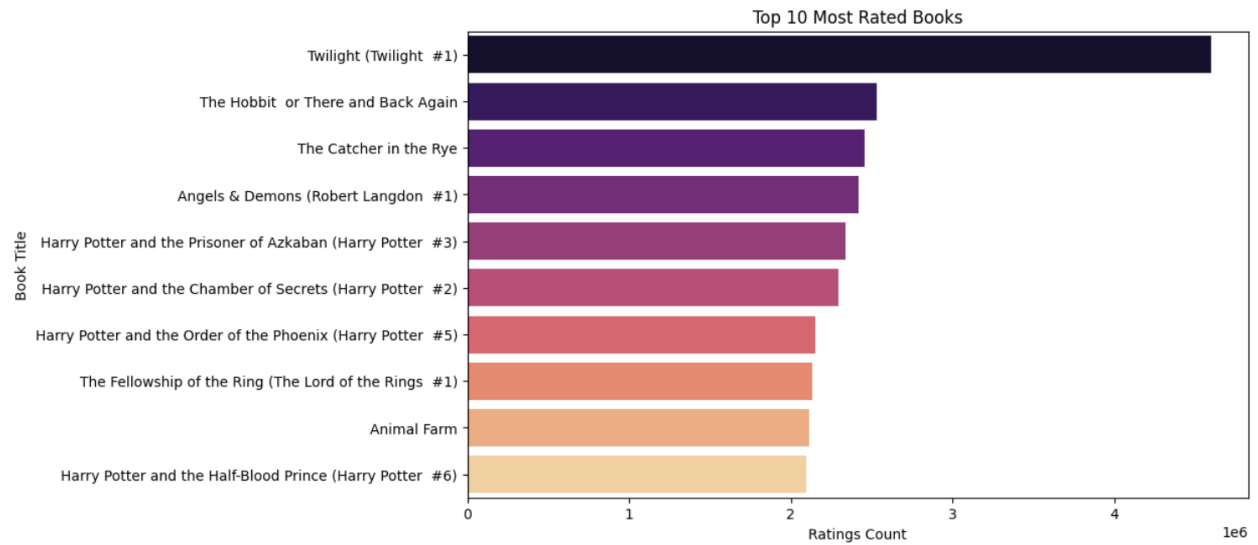
Recall for not.fully.paid=1: 0.8178993163455562



Cross-validation scores: [0.87067961 0.87572816 0.87995338 0.87101787 0.86674437]
Mean Accuracy: 0.8728246769411818
Standard Deviation: 0.004562556701735428

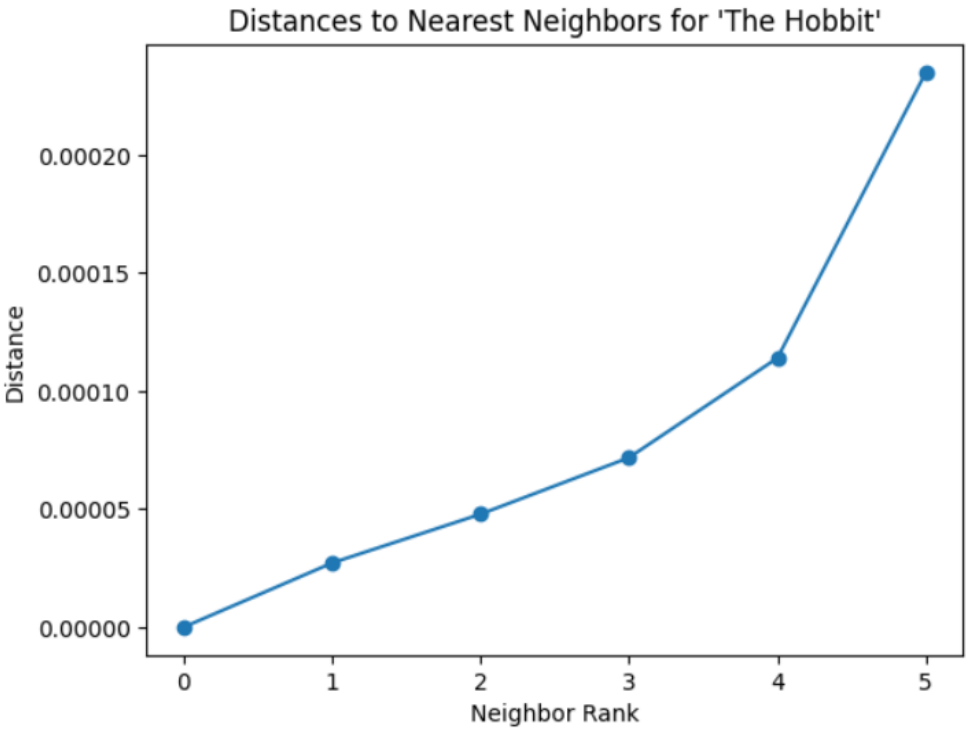
Task02: Book Recommender





```
print(BookRecommender("The Hobbit"))
```


title	authors	average_rating	ratings_count
The Hobbit	J.R.R. Tolkien	4.27	3213
The Voyage of the Jerle Shannara Trilogy (Voyage of the Jerle Shannara #1-3)	Terry Brooks	4.27	3088
Tsubasa: RESERVoIR CHRONICLE Vol. 8	CLAMP/William Flanagan	4.27	2993
The Complete Plays	Sophocles/Paul Roche	4.27	2883
Ariel: The Restored Edition	Sylvia Plath/Frieda Hughes	4.27	3737
The Forbidden (Vampire Huntress #5)	L.A. Banks	4.27	2133



5. Conclusion:

Through these tasks, I gained practical experience in applying different machine learning algorithms to real-world problems. In the loan default prediction task, I learned the importance of handling class imbalance and avoiding overfitting by comparing Decision Trees and Random Forests. In the book recommender task, I explored collaborative filtering with k-NN and understood the challenges of sparse data and scalability. Overall, these tasks improved my understanding of model selection, evaluation metrics, and preprocessing techniques.

Future Recommendations

For future work, more advanced models such as Gradient Boosting or Neural Networks could be applied to improve predictive performance. In the recommender system, incorporating hybrid approaches (content + collaborative filtering) and dimensionality reduction techniques like SVD could address sparsity and scalability issues. Additionally, deploying these models in a real-world application would provide deeper insights into their usability and effectiveness.

6. References:

- <https://www.youtube.com/>
- <https://www.youtube.com/@campusx-official>
- <https://developers.google.com/machine-learning/gan>